GenSIE @ IberLEF 2026: Schema-Guided Information Extraction with Small Language Models in Spanish

Alejandro Piad Morffis[1], Yudivian Almeida Cruz[1], Suilan Estévez Velarde[1], Isabel Espinosa Zaragoza[2], María Miró Maestre[3], Lucía Sevilla Requena[3], Alba Pérez Montero[2] and Ernesto Estevanell Valladares[1,2]

[1]*GIA, University of Havana, Cuba*
[2]*GPLSI, University of Alicante, Spain*
[3]*CENID, University of Alicante, Spain*

Abstract

GenSIE is the IberLEF 2026 shared task on schema-guided information extraction from Spanish text using small language models. Participants build CPU-side agents that consume a `(text, instruction, JSON schema)` triple and emit a JSON object that conforms to the schema while staying faithful to the text — including the explicit obligation to return `null` for fields the text does not support. We contribute a 271-instance Spanish benchmark spanning eight development domains and sixteen schemas — with a held-out test set that adds a ninth domain, Research — built by a semi-automatic agentic curation pipeline with three human gate points, and an evaluation framework that pairs micro-F1 over flattened schemas with a multi-model gap-closed metric. The leaderboard rewards how much of the baseline-to-perfect error each system removes, averaged across the two published models. Eleven teams submitted thirty-five pipelines. The official evaluation was run on 125 gold test instances — the full 145-instance set less a 20-instance drop-set of schemas that could not be reliably scored on the constrained-decoding inference stack — against per-model no-think baselines (Gemma 4 E4B F1=0.7895, Qwen3-14B F1=0.7805). DRILLER tops the leaderboard, closing 21.6% of the baseline-to-perfect gap with a RAG pipeline family — its self-consistency ensemble tops Gemma (F1=0.8285, think mode) while its single-call variant tops Qwen (F1=0.8346); the field spans roughly 0–22% gap closed, with a tight cluster of top systems — a schema-adaptive pipeline (Krishan, 12.0%) and a guard-first repair pipeline (CodeStrange `combo_guard_react_repair_ref`, 10.0%) — landing at ~0.80–0.83 F1 on the clean instances; a paired-bootstrap test finds these top systems statistically indistinguishable from one another. As a methodological note, we find that Gemma 4 E4B's thinking mode dramatically shifts the effective baseline (F1=0.4249 thinking-ON vs 0.7895 no-think on the 125 scored instances), which complicated cross-team comparison until we standardised on the no-think 125-instance floor.

Resumen: GenSIE es la tarea compartida de IberLEF 2026 sobre extracción de información guiada por esquemas a partir de texto en español usando modelos de lenguaje pequeños. Los participantes construyen agentes que consumen una tripleta `(texto, instrucción, esquema JSON)` y emiten un objeto JSON conforme al esquema y fiel al texto, incluyendo la obligación explícita de retornar `null` para campos que el texto no sustenta. Contribuimos un benchmark de 271 instancias en español que abarca ocho dominios de desarrollo y dieciséis esquemas —con un conjunto de prueba reservado que añade un noveno dominio, Research— construido mediante un pipeline de curación semi-automático con tres puntos de control humano, y un marco de evaluación que combina micro-F1 sobre esquemas aplanados con una métrica de brecha cerrada multi-modelo. Once equipos enviaron treinta y cinco pipelines. La evaluación oficial se ejecutó sobre 125 instancias doradas —el conjunto completo de 145 menos un descarte de 20 instancias cuyos esquemas no pudieron puntuarse de forma fiable en la pila de inferencia con decodificación restringida— frente a líneas base sin modo de razonamiento (Gemma 4 E4B F1=0.7895, Qwen3-14B F1=0.7805). DRILLER encabeza la clasificación cerrando el 21,6% de la brecha entre la línea base y la extracción perfecta mediante una familia de pipelines RAG —su conjunto por autoconsistencia lidera en Gemma (F1=0,8285, modo razonamiento) y su variante de una sola llamada lidera en Qwen (F1=0,8346)—; el campo abarca aproximadamente entre 0% y 22% de brecha cerrada, con un grupo compacto de sistemas punteros —un pipeline adaptativo al esquema (Krishan, 12,0%) y un pipeline de reparación con validación previa (CodeStrange `combo_guard_react_repair_ref`, 10,0%)— situándose en ~0,80–0,83 F1 sobre las instancias limpias; un test de bootstrap pareado halla estos sistemas top estadísticamente indistinguibles entre sí. Como nota metodológica, observamos que el modo de razonamiento de Gemma 4 E4B desplaza drásticamente la línea base efectiva (F1=0.4249 con razonamiento frente a 0.7895 sin él, sobre las 125 instancias evaluadas).

Keywords

Information Extraction, Schema-Guided Extraction, Small Language Models, JSON Schema, Spanish NLP, Shared Task

IberLEF 2026, September 2026, León, Spain
apiad@matcom.uh.cu (A. P. Morffis)

1. Introduction

The deployment gap between frontier and edge-tier language models is now the dominant cost in applied structured information extraction. A 70B-parameter model in a managed API extracts reliably from a wide variety of unseen schemas; the same task on a 3B–14B model that fits an on-premise CPU box requires schema-handling tricks, retrieval, and self-verification scaffolding that nobody has standardised. Most published benchmarks for structured extraction either fix the schema in advance, score only English text, or assume access to a frontier model — none of which match the deployment reality for Spanish-language organisations that want to keep data on their own hardware.

GenSIE is built around the hypothesis that the small-model tier is more viable than the literature suggests, provided participants are free to invest in agentic scaffolding around the model rather than the model itself. The task gives them a Spanish text, a natural-language instruction, and a JSON schema, and asks for a schema-conformant object. The schema is unknown at training time and may not appear at all in the development data — roughly half of the test schemas are deliberately new. The system runs on a CPU-only container and calls an organiser-hosted inference server that exposes several open-source models in the 3B–14B parameter range, with a subset held out so that single-model tuning is penalised structurally.

This setup tests the limits of prompt engineering, retrieval augmentation, schema enrichment, and verify-revise loops. Specifically, it measures how well these techniques guide a small model through a previously unseen Spanish schema under explicit token and time budgets. A shared task provides the necessary methodological rigour by evaluating all approaches against a unified harness and a single dataset — a standardisation that current literature on structured extraction frequently lacks.

We contribute four artefacts: (1) a 271-instance Spanish extraction benchmark with sentence-level grounding annotations and complexity-tagged schemas across nine domains; (2) a semi-automatic agentic curation pipeline in which agents do roughly 95% of the labour and humans gate three decisive points; (3) a multi-model gap-closed evaluation that is comparable across models with different absolute ceilings; and (4) the first IberLEF round of small-model schema-guided extraction at the 3B–14B tier with multi-model held-out evaluation.

The rest of the paper is organised as follows. §2 defines the task. §3 introduces the complexity taxonomy that drives dataset balance. §4 describes the agentic curation pipeline that produced the gold instances. §5 specifies the evaluator and the gap-closed metric. §6 reports the reference baseline. §7 catalogues the eleven participating teams and their strategies. §8 reports the official test-set results, including the excluded-instance methodology and the final leaderboard. §9 discusses the signals from the evaluation. §10 positions GenSIE against related work. §11 concludes and sketches the next-year direction.

2. Task Definition

GenSIE is a single-task benchmark. Each instance is a triple `(text, instruction, target_schema)`. The text is a Spanish-language passage drawn from sources that range across Wikipedia, news, regulatory filings, clinical-trial registries, and review aggregators. The instruction is a short natural-language sentence that frames the extraction goal — for instance, "extract the medication and the trial outcome, classifying outcome as positive above 90% efficacy, negative below 70%, inconclusive otherwise". The target schema is a JSON Schema document in the OpenAPI 3.0 subset. The system must emit a JSON object that validates against the schema and that faithfully reflects the text.

A central design choice is the null-as-ground-truth semantic. Some schema fields are deliberately asked of texts that do not support them; the correct answer in that case is `null`, and a fabricated value receives zero credit, regardless of how plausible it is. This penalises parametric hallucination — the failure mode in which the model fills a slot from its prior rather than from the source. The frequency of null targets is kept intentionally low so that defaulting to empty output is never a winning strategy, but their presence is enough to deter pipelines that simply ignore grounding.

The hardware contract is asymmetric by design. Participant code runs in a CPU-only Docker container with no public-internet access. The container's only permitted outbound connection is to an organiser-hosted OpenAI-compatible inference server that runs the language models on 8× A100 GPUs. The container must expose `GET /info` and `POST /run` HTTP endpoints, and completions must be requested with `stream: false` so the evaluator can read exact token usage from each response. The runtime model name is passed at request time — participants must not assume any specific model.

Two further design decisions distinguish GenSIE from prior structured-extraction benchmarks. First, the zero-shot schema policy: roughly half of the 145 test instances use schemas that do not appear anywhere in the development data. The other half use schemas that are present in development but extracted from previously unseen source documents. This forces submissions to consume the schema dynamically at inference time rather than memorise field names. Second, the task is partitioned across nine domains and sixteen schemas — eight are present in the development set; the ninth, Research, appears only in the held-out test set, consistent with the zero-shot-schema policy above:

Domain	Schemas
Cultural	Literature, Monuments, Movie Reviews
Environmental	Ecology
General	Disasters
Legal	Contracts, Judicial, Legislation
Lifestyle	Recipes, Travel
Medical	Diseases, Drug Safety, Health News
Research	Paper Argumentation
STEM	Astronomy
Technical	Software

The dataset comprises 271 silver development instances and 145 gold test instances, with the test split balanced across both seen-schema-new-text and unseen-schema partitions.

3. Schema Complexity Dimensions

Not all schemas are equally hard. To make dataset balance auditable and to enable post-hoc analysis of where systems fail, every schema in GenSIE carries a structured complexity profile attached via the `@complexity` decorator on its Pydantic class. Five dimensions are tracked, plus an overall aggregate level.

Structural depth (1–4) measures how nested the schema is. A depth-1 schema is flat: a handful of top-level fields. A depth-4 schema has nested objects containing lists of nested objects with their own enums and references.

Semantic distance (1–4) measures how far the extraction is from verbatim copy. Distance 1 is verbatim: the value appears as a span in the text. Distance 4 is semantic mapping: the value is an enum the model must infer, such as classifying "94.1%" as `POSITIVE` because the instruction defines the cutoff at 90%.

Information dispersion (1–4) measures how scattered the extractable information is. Dispersion 1 keeps everything in a single sentence; dispersion 4 spans the whole document and requires the model to reconcile facts that appear paragraphs apart.

Constraint rigidity (1–4) measures how tightly the schema constrains its values. Rigidity 1 admits free-form strings; rigidity 4 enforces an enumerated value set, sometimes with format strings or numeric ranges.

Grounding difficulty (1–3) measures evidence density. Grounding 1 is dense and direct; grounding 3 is mixed, where some fields have clear textual support and others must be returned as `null` for the same instance.

These dimensions roll up into a single overall level from `L1` (trivial) to `L10` (extreme), via a rule-based mapping that prioritises distance and dispersion over depth — a flat schema with semantic-distance-4 fields is treated as harder than a deeply nested schema of verbatim spans.

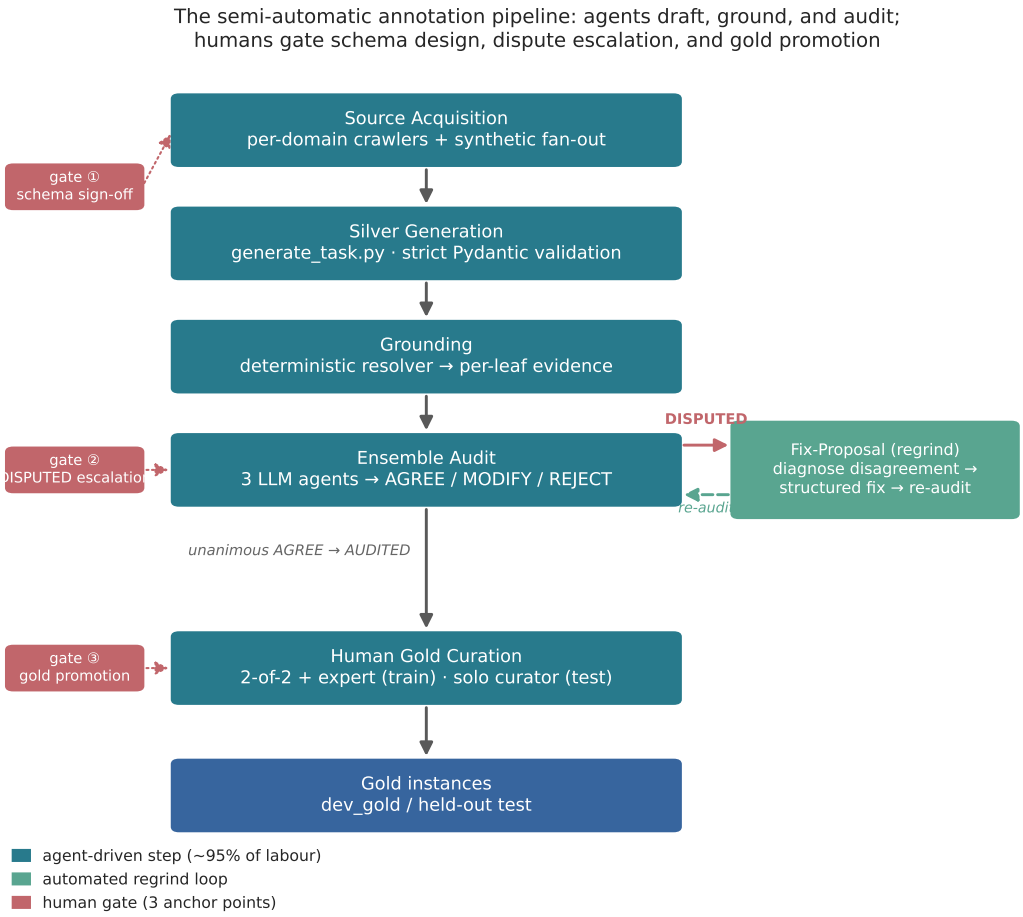

Figure 1: The semi-automatic annotation pipeline. Agents perform roughly 95% of the labour — sourcing text, drafting schema-conformant silver instances, grounding every leaf to source evidence, and auditing by a three-agent ensemble — while a DISPUTED regrind loop repairs disagreements automatically. Three human gates anchor the parts that cannot be delegated: schema sign-off, dispute escalation, and silver-to-gold promotion.

The 271-instance training/dev silver set spans eight of the benchmark's nine domains (the ninth, Research, appears only in the held-out test set). Grounding coverage is high: 256 of 271 instances (94%) are fully grounded. The grounding breakdown by kind is: extractive 44.5%, inferred 28.3%, paraphrased 6.1%, absent 21.0%. Complexity is concentrated in the L4–L5 range — the distribution across levels, over the 256 fully-grounded instances, is L1(9), L2(16), L3(7), L4(58), L5(86), L6(10), L7(26), L8(16), L9(15), L10(13) — with average dimension scores of depth 2.21, dispersion 2.37, semantic distance 2.54, grounding 2.20, and rigidity 2.72.

4. Dataset Construction: The Agentic Loop

The gold instances were built by a semi-automatic curation pipeline. We deliberately frame it as semi-automatic rather than agentic or human-annotated: agents do roughly 95% of the labour — sourcing, drafting, grounding, auditing, fixing — and humans gate three decisive points. Neither extreme would have worked. A fully automated pipeline cannot guarantee grounding fidelity on a benchmark whose value hinges on grounding fidelity. A fully manual pipeline does not scale to sixteen schemas across nine domains with per-leaf evidence annotations. Figure 1 sketches the full flow.

4.1. Source Acquisition

Per-domain crawlers populate `data/raw/<domain>/<subdomain>/`. The crawlers target sources with stable, freely licensed Spanish text: Wikipedia and Wikinoticias for general and cultural content, the Spanish Boletín Oficial del Estado (BOE) for legal text, the CIMA registry for drug safety, Espinof for movie reviews, and a handful of curated open-access scientific outlets for medical and STEM coverage. Each source carries an `index.json` provenance record with URL, retrieval date, and license. When a domain × complexity cell is undercovered, a synthetic-raw agentic fan-out generates native-Spanish documents tailored to a target schema and complexity level; these are then treated like any other source by the rest of the pipeline.

4.2. Silver Instance Generation

`scripts/generate_task.py` is the single write authority for silver instances — no agent generates silvers directly. The script makes one LLM call per `(source, schema)` pair, asks for a structured JSON output, and validates the response strictly against the target Pydantic schema before writing. Anything that fails Pydantic validation is rejected without retry: the resulting silver corpus is therefore guaranteed schema-conformant by construction, even before any grounding work begins.

4.3. Grounding

For every silver instance, a deterministic resolver post-processes the output and emits a sibling `<task_id>.grounding.json` file. Each leaf value gets a breadcrumb with four fields: `kind`, one of `extractive`, `inferred`, `paraphrased`, or `absent`; `quote`, the supporting span when `kind` is extractive or paraphrased; character offsets into the source text; and `reasoning`, a short natural-language justification. The grounding file is what makes downstream auditing reliable — auditors check evidence, not just the JSON output. A leaf that the silver generation marked as `null` produces an `absent` breadcrumb with the model's reasoning for why the text does not support the field.

4.4. Ensemble Audit

Three independent LLM agents audit each grounded leaf in parallel. Each agent receives the source text, the schema, the proposed value, and the grounding breadcrumb, and returns `AGREE`, `MODIFY`, or `REJECT` with a free-text rationale. A leaf is marked AUDITED only when all three agents return `AGREE`; any other combination is marked DISPUTED. Audits are signed by agent identity and append-only — re-audits accumulate rather than overwrite. Unanimous agreement across three independent agents is treated as the lower bound for promoting a silver to gold without human inspection of that specific leaf.

4.5. Fix-Proposal Pipeline

DISPUTED instances enter the regrind loop. A diagnostic subagent reads the three audit verdicts, identifies the disagreement pattern (value error, grounding kind mismatch, missing evidence, hallucinated value, or schema misuse), and emits a structured fix proposal. A deterministic apply script archives the prior state under a versioned path and replaces the grounding in place. The instance returns to the audit queue and is re-audited; the loop terminates when the ensemble agrees, when the diagnostic subagent escalates to a human gate, or when a stop counter is hit.

4.6. Human Gold Curation

The annotation UI is a Streamlit application with two modes. The 2-of-2 + expert mode is used for the training silver pool: two annotators each finalise the instance independently, conflicts route to an expert annotator who resolves them, and only matching final outputs promote into `data/dev_gold/`. The solo curator mode is used for the held-out test set: one expert annotator finalises each instance with

a single ACCEPT or MODIFIED action, with the audit ensemble's verdict and the diagnostic subagent's fix proposal pre-loaded as context.

Three human gate points anchor the pipeline: silver-to-gold promotion, DISPUTED escalation (when the regrind loop fails to converge), and schema-level acceptance (a human signs off on the schema definition itself before any instance is generated against it).

4.7. What Is Not Automated

Four decisions remain under human authorship: (1) schema design, (2) domain selection, (3) final test-set curation decisions, and (4) the definition of extraction intent — what "outcome" means in a clinical-trial schema, what counts as POSITIVE in a movie-review enum.

5. Evaluation Framework

The evaluator has three layers stacked together: a per-instance micro-scored flattened-schema comparison, a multi-model averaging step, and a gap-closed normalisation that turns absolute F1 into a comparable improvement-over-baseline metric.

5.1. Flattened Schema Scoring

Both the gold answer and the system output are first flattened to dot-notation key-value pairs by a transformation we call $\Phi(J)$. A nested object `{"event": {"city": "Madrid", "details": {"attendees": 500}}}` becomes `{"event.city": "Madrid", "event.details.attendees": 500}`. List items are keyed by position: `tags: ["A", "B"]` becomes `{"tags.0": "A", "tags.1": "B"}`.

For each key present in either object, we compute a per-field similarity score. Rigid-type fields (numbers, dates, booleans, enums) score exact match: any deviation, including casing differences on enum strings, drops the field to zero. Free-text fields score a blended cosine-similarity-over-embeddings (multilingual MPNet (Reimers and Gurevych 2019)) and token-overlap, with $\alpha \approx 0.7$ favouring the semantic component. Lists score under order-independent greedy bipartite matching followed by Jaccard normalisation.

The null-as-ground-truth contract:

Gold	System	Score
`"field": "value"`	`"field": null`	0.0 (hallucinated null)
`"field": null`	`"field": "value"`	0.0 (hallucinated value)
`"field": null`	`"field": null`	1.0 (correct null)
missing key	`"field": "value"`	ignored, but increases denominator

Worked example. Gold `{"name": "Madrid", "year": 1982, "host": null}` vs system `{"name": "Madrid", "year": 1982, "host": "FIFA"}`: `name` exact match (1.0), `year` exact match (1.0), `host` hallucinated value against null gold (0.0). TPS = 2.0, precision and recall both 2/3, F1 = 2/3.

5.2. The Gap-Closed Metric

Raw micro-F1 is not the primary leaderboard. For each evaluation model, let F1_b be the official baseline's micro-F1 and F1_s the system's micro-F1. The gap-closed score is:

$$\text{GapClosed} = \max(0, (\text{F1\_s} - \text{F1\_b}) / (1 - \text{F1\_b}))$$

A baseline at 0.60 leaves 40 points of error on the table. A system at 0.70 has closed 25% of that gap; a system at 0.80 has closed 50%. The final per-team score is the average of GapClosed across all evaluation models, taking each team's best-performing pipeline per model.

Two-team worked example. On model M, baseline 0.60: Team A scores 0.70 ⟶ gap closed 0.25; Team B scores 0.80 ⟶ gap closed 0.50. On model M', baseline 0.80: Team A scores 0.85 ⟶ gap closed

0.25; Team B scores 0.82 → gap closed 0.10. Team A's average: 0.25. Team B's average: 0.30. Team B ranks higher.

Why gap-closed, not raw F1? First, +5 F1 from 70→75 is much easier than +5 from 90→95 — diminishing returns. Second, raw F1 is hard to compare across models with different absolute ceilings. Gap-closed is normalised per model, so it is comparable and averageable across models, and it directly rewards what GenSIE is about: pulling a small model as close as possible to frontier-model performance on a previously unseen schema. The normalized-improvement shape — the fraction of baseline-to-ceiling headroom a system closes — is itself well established (cf. human-normalized scores in reinforcement learning (Mnih et al. 2015) and the human-baseline-gap framing of SuperGLUE (Wang et al. 2019)); our adaptation is the per-model normalization averaged over several held-out models, which we position in §10.

5.3. Multi-Model Evaluation

Each pipeline is run independently against several models. The published models, named ahead of time so participants can iterate against them, are Gemma 4 E4B (Gemma Team, Google DeepMind 2026) (primary required model) and Qwen3-14B (Qwen Team 2025) (secondary). A second set of held-out models is not disclosed before results; they are all small (≤14B parameters), open-source, and from different model families than the published set, so that single-model tuning is structurally penalised.

Note: the models used for the preliminary dry run (`llama-3.2-3b-instruct` and others) were development-phase models, not part of the formal evaluation set.

5.4. Resource Budgets

Two soft budgets are enforced, both averaged over the test set: - Token budget: 32K total tokens/instance (input + completion + any reasoning, retries, self-corrections). Individual instances may reach 2× budget if the run-wide average stays ≤32K. - Wall-time budget: 60 seconds/instance, counting only user-code time (inference wait time not counted). Total ≈ 145 minutes per pipeline per model.

These are soft averages, not hard caps. Systematic, sustained overshoot is penalised post-hoc; occasional 5–10% exceedance is not.

6. Reference Baseline

The reference baseline is `gensie.baseline.OfficialParticipant`, a factory that registers a single pipeline named `baseline` whose implementation is `BasicAgent`. The agent issues one OpenAI structured-outputs call per instance, passing the schema verbatim as the `json_schema` response format with `strict: true`. There are no retries, no RAG index, no schema enrichment, no self-consistency, no ensemble fan-out.

This pipeline is the floor of the leaderboard by construction: the gap-closed metric assigns zero credit to anything at or below the baseline. On the 40-instance starter kit (preliminary dry run), the reference baseline scored 0.6051 micro-F1 on `llama-3.2-3b-instruct`. The official baseline over the scored 125-instance test set (no-think configuration, median of three runs — see §8.1) is F1=0.7780 on Gemma 4 E4B and F1=0.7542 on Qwen3-14B; these are the per-model floors against which every system's gap-closed score is computed.

7. Participants and Strategies

7.1. Registration Overview

Thirteen teams registered for GenSIE 2026 across Cuba, Spain, Colombia, and Mexico. Of these, two did not complete their submission — one withdrew before the deadline and one did not grant repository access — leaving eleven teams as confirmed active participants:

#	Team	Institution	Country
2	VerbaNexAI Lab	Universidad Tecnológica de Bolívar	Colombia
7	FranRodrigo	UNED	Spain
8	CodeStrange	Universidad de La Habana / independent	Cuba
9	MOLD	Universidad de La Habana	Cuba
10	DRILLER	Universidad de La Habana	Cuba
11	IÑIGOIKO	Universidad Pública de Navarra	Spain
12	GRADIANT	GRADIANT	Spain
16	JSONautas	UHO / UO / UCI	Cuba
19	ExtractUH (SEsml)	Universidad de La Habana	Cuba
20	Krishan Heredia	Independent	Cuba
21	UC3M_NLP	Universidad Carlos III de Madrid	Spain

These eleven teams collectively submitted thirty-five pipelines (MOLD and GRADIANT declared four each, one over the three-pipeline limit).

7.2. Strategy Classification

We classify every submitted pipeline along five orthogonal axes. Approach is the control flow: a single *direct-prompt*, a *chain-of-thought* that reasons in natural language before emitting JSON, a *multi-extractor* that splits the schema across concurrent per-field calls, or an *agentic-loop* that drafts, critiques, and repairs over several turns. Knowledge is the external context injected before extraction: *none*, a *static* curated example pool, or *RAG-emb* embedding retrieval over an indexed corpus. Self-consistency is how multiple samples are reconciled: *none*, a *verify-revise* critique pass, majority *vote*, or *k-sampling*. Schema is how the JSON Schema is presented to the model: *raw*, *enriched* with natural-language descriptions and type hints, or *dynamic* (labels derived at runtime from the schema itself). Calls counts the LLM invocations per instance. The full classification of all 35 pipelines follows; Figure 2 distils it to one flagship pipeline per team, ordered by final rank, so that strategy can be read against standing.

Team	Pipeline	Approach	Knowledge	Self-cons.	Schema	Calls	Ensemble
CodeStrange	combo_guard_react_repair_ref	agentic-loop	none	verify-revise	enriched	2–3	no
CodeStrange	grounded	agentic-loop	RAG-emb	none	enriched	2–3	no
CodeStrange	combo_guard_list_spacy_ref	direct-prompt	static	none	enriched	1	no
DRILLER	enriched-schema-rag	direct-prompt	RAG-emb	none	enriched	1	no
DRILLER	enriched-inline-reasoning-rag	chain-of-thought	RAG-emb	none	enriched	1	no
DRILLER	mixed-extractors-self-consistency-rag	multi-extractor	RAG-emb	vote	enriched	4+	yes
FranRodrigo	structured	direct-prompt	none	none	enriched	1	no
FranRodrigo	cot	chain-of-thought	none	none	raw	1	no
FranRodrigo	verify	agentic-loop	none	verify-revise	enriched	2–3	no
GRADIANT	baseline	direct-prompt	none	none	raw	1	no
GRADIANT	stable	agentic-loop	none	k-sampling	enriched	4+	no

Team	Pipeline	Approach	Knowledge	Self-cons.	Schema	Calls	Ensemble
GRADIANT	experimental	multi-extractor	none	none	dynamic	2–3	no
GRADIANT	limited	multi-extractor	none	none	dynamic	2–3	no
Iñigo	e226	direct-prompt	static	none	enriched	1	no
Iñigo	e231	direct-prompt	none	none	raw	1	no
Iñigo	e232	direct-prompt	static	none	enriched	1	no
JSONautas	prompt_eng	direct-prompt	none	none	raw	1	no
JSONautas	rag_domain	direct-prompt	RAG-emb	none	raw	1	no
JSONautas	grammar_cd	chain-of-thought	none	none	enriched	1	no
Krishan	spanish_fewshot	direct-prompt	none	none	enriched	1	no
Krishan	english_fewshot	direct-prompt	none	none	enriched	1	no
Krishan	schema_dynamic	direct-prompt	none	none	dynamic	1	no
MOLD	baseline	direct-prompt	none	none	raw	1	no
MOLD	mira	agentic-loop	none	verify-revise	enriched	2–3	no
MOLD	vigil	agentic-loop	RAG-emb	verify-revise	enriched	4+	no
MOLD	arcane	agentic-loop	RAG-emb	verify-revise	enriched	4+	no
SEsml	extraction	chain-of-thought	none	verify-revise	enriched	2–3	no
SEsml	hybrid_cot	chain-of-thought	none	none	enriched	1	no
SEsml	adaptive	chain-of-thought	RAG-emb	none	dynamic	1	no
UC3M_NLP	prompted	direct-prompt	none	none	raw	1	no
UC3M_NLP	eagle	multi-extractor	none	none	dynamic	4+	no
UC3M_NLP	rsa	agentic-loop	none	vote	raw	4+	no
VerbaNexAI	smart_orchestrator	multi-extractor	RAG-emb	none	enriched	1–2	yes (routing)
VerbaNexAI	precision_master	direct-prompt	none	none	enriched	1	no
VerbaNexAI	parallel_extractor	multi-extractor	none	none	enriched	2–3	no

Aggregate: direct-prompt 15/35, agentic-loop 8, chain-of-thought 6, multi-extractor 6. External knowledge: none 23, RAG-emb 9, static 3. Self-consistency: none 26, verify-revise 6, vote 2, k-sampling 1. Schema: enriched 22, raw 8, dynamic 5. Calls: 1 for 20, 1–2 for 1, 2–3 for 8, 4+ for 6.

The space is dominated by *single-pass, context-enrichment* strategies: well over half of pipelines make a single call, most enrich the schema before prompting, and a quarter add embedding retrieval — the levers §8.6 finds actually pay off. The multi-step end (agentic verify-revise loops, four-trial ensembles, per-field concurrent extraction) was explored by a minority and, as the results show, is where compute was most often spent without return. Two design axes went essentially unexplored and are open for next year: fine-tuning or distillation — foreclosed by the no-training policy and the hosted-inference harness (§9.6) — and genuine cross-model routing beyond VerbaNex's domain-based dispatcher.

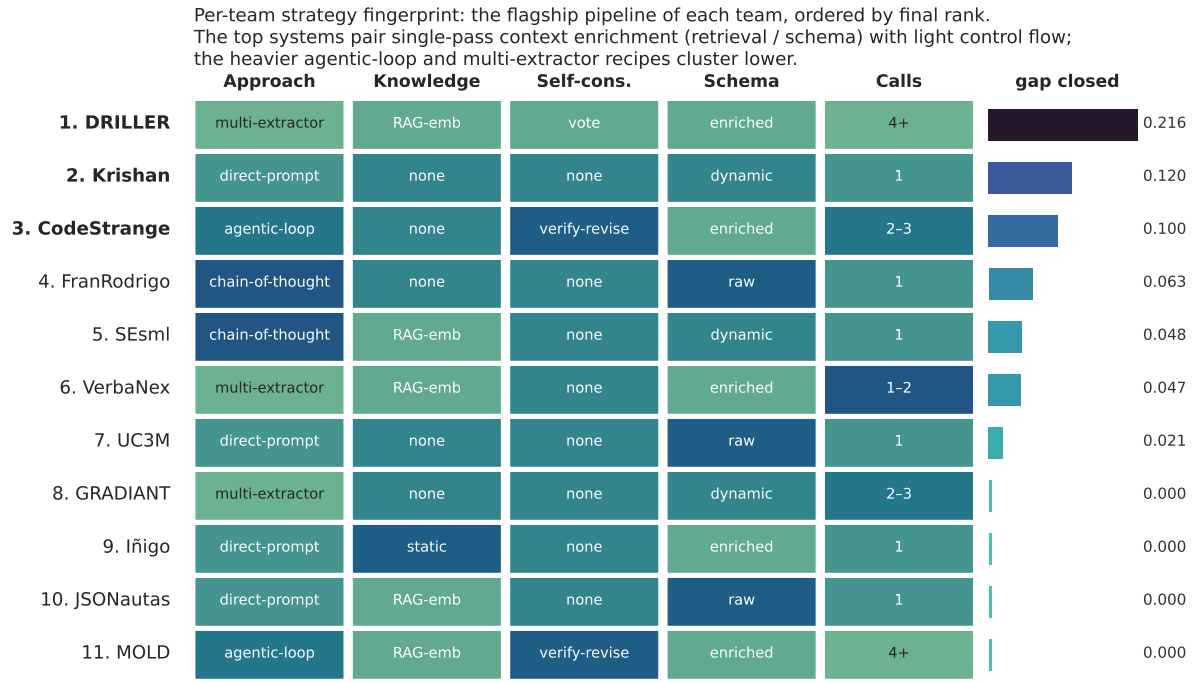

Figure 2: Per-team strategy fingerprint: the flagship pipeline of each team along the five axes above, ordered top-to-bottom by final leaderboard rank, with the gap-closed score as a bar on the right. The top of the board pairs single-pass context enrichment (retrieval and/or schema enrichment) with light control flow, while the heavier agentic-loop and multi-extractor recipes concentrate lower down.

7.3. Per-Team Briefs

Each brief describes the team's approach and its behaviour during the development-phase dry run (`llama-3.2-3b-instruct`, 40-instance starter kit) and the container health-check phase of the formal sweep. Any F1 quoted in a brief is a dry-run number on that small set unless stated otherwise; the official test-set results (Gemma 4 E4B / Qwen3-14B, 125 scored instances) are in §8 and were obtained by a mix of organiser re-runs, team self-evaluations, and the June formal sweep, so a pipeline that failed a container health-check here may still carry a scored cell in §8 recovered from a later re-run.

CodeStrange (Leynier Gutiérrez González, independent, Mexico City; Carlos Bermúdez Porto, independent, Havana; Roberto Martí Cedeño, Universidad de La Habana) submitted three pipelines built around guard-first drafting and rule-based validation. The flagship `combo_guard_react_repair_ref` issues a temperature-zero draft, classifies issues against a five-kind validator (missing-required, empty-required-array, low-recall-list, suspicious-null, unsupported-non-null), and fires up to two repair calls with `guard_fix=True` on escalation, in the style of ReAct-style act-observe loops (Yao et al. 2023). A `grounded` pipeline adds fastembed-based retrieval over truncated source tails for heavy-output schemas. A `combo_guard_list_spacy_ref` pipeline routes per-field roles through a Spanish spaCy (Honnibal et al. 2020) pipeline before a single LLM call. The team's self-evaluation on the full 145-instance test set (§8.3) shows `combo_guard_react_repair_ref` as the strongest pipeline by a substantial margin.

DRILLER: Dynamic Reasoning for Information Labeling and Literal Evidence Retrieval (Ariel González Gómez, Daniel Valdés Pérez, Universidad de La Habana) submitted three pipelines, all using fastembed retrieval over a pre-indexed dev-field corpus (Lewis et al. 2020): a single-call direct-prompt with two retrieved few-shot examples; a chain-of-thought wrapper (Wei et al. 2022) over the same call; and the only true ensemble in the field, which fans out across four interleaved internal extractions then aggregates via a schema-aware self-consistency aggregator (Xuezhi Wang et al. 2023).

ExtractUH (SEsml): Schema-guided Extraction with Small Language Models (Alejandro Beltrán, Daniel Toledo, Facultad de Matemática y Computación, Universidad de La Habana) submitted three

pipelines built around a local SchemaOptimizer that pre-processes the JSON Schema. `extraction` runs a DraftEngine to produce a `<Thinking>+<Draft>` intermediate, then a constrained-decoding pass. `hybrid_cot` collapses the two passes into a single CoT call. `adaptive` adds a fastembed-based SchemaAnalyzer with k=15 weighted voting and a PromptAssembler within a 1400-character budget.

FranRodrigo (Francisco Javier Rodrigo Ginés, UNED) submitted three pipelines. `structured` uses a schema-to-natural-language guidelines converter with `json_schema` strict mode. `cot` issues a single chain-of-thought prompt asking the model to quote the relevant passage before the value, then extracts JSON via regex. `verify` is a two-pass extract-then-critique loop.

GRADIANT (Álvaro Bueno Sáez, Héctor Cerezo Costas, David Sueiro Durán, GRADIANT) submitted four pipelines — one over the limit. `baseline` mirrors the reference agent. `stable` pre-normalises text with two analysis calls (temperature=0.1), then fires up to five sampling-based extraction calls — or a single verbatim-grounding call for simple field types — and consolidates via a verification pass. `experimental` is a plan-and-execute pipeline that categorises each schema field and dispatches per-category strategies. `limited` is a time-gated subclass of `experimental` with a τ=40s threshold for activating chain-of-thought reasoning (reasoning_effort="low"); it applies CoT only to fixed_entities, soft_entities, and complex field categories, and is compatible with thinking-capable model families (Qwen3, QwQ, DeepSeek-R1, O-series).

Iñigo / IÑIGOIKO (Iñigo Goikoetxea Fanega, Universidad Pública de Navarra) submitted three single-call direct-prompt pipelines differing in few-shot retrieval. `e226` selects from an external 28MB curated example pool using an enriched schema-type matcher. `e231` is a zero-shot variant. `e232` hardens `e226` with a robust hybrid selector (structural top-N reranked by TF-IDF) and gating on confidence. All three rely on a `heuristic_fix_v6_3` post-processor.

JSONautas (Yisel Clavel Quintero, University of Holguín; Dionis López Ramos, University of Oriente; Derek Naharai Herrera Domínguez, UCI) submitted three pipelines: a raw-schema prompt-engineering pipeline with XML semantic zones and strict null rules; a domain-routed RAG pipeline over a GloVe 6B.50d-indexed (Pennington et al. 2014) glossary corpus partitioned by domain, explored in five architectural variants (flat cosine retrieval, domain-hierarchical with per-domain embedding matrices, system-prompt context injection, batch-vectorised domain-GloVe retrieval, and asynchronous FAISS/HNSW-indexed (Johnson et al. 2021) chunking with FastEmbed/BGE-small (Xiao et al. 2023)); and a grammar-constrained-decoding pipeline using `outlines.from_openai` (Willard and Louf 2023) with a CoT `_reasoning` field injected into the schema and low temperature (0.0–0.2) for deterministic output.

Krishan Heredia (Eötvös Loránd University, Hungary) submitted three direct-prompt pipelines with three-retry validator-driven correction, using temperature=0.5 (selected after testing the full 0–1 range). `spanish_fewshot` uses Spanish prompts and hardcoded NER-label few-shot examples. `english_few shot` is the same architecture in English. `schema_dynamic` derives labels at runtime from the schema enum order.

MOLD (Roberto García Rodríguez, Danilo Guerrero Montero, Universidad de La Habana) submitted four pipelines — one over the limit. `baseline` is a single strict json_schema call. `mira` is a two-pass unconstrained-analysis-then-strict-extraction loop. `vigil` adds gated FAISS+MiniLM (Wang et al. 2020) retrieval (k=3, threshold 0.55) and an Architect-module reasoning hint. `arcane` further double-gates the retrieval with an audited synthetic example (cosine $\geq$ 0.70).

UC3M_NLP (Pablo Martín Berná, Mauro García Lorenzo, Carlota Ontangas Serón, Lucía Velasco González, Universidad Carlos III de Madrid) submitted three pipelines. `prompted` is a single-call Spanish-system-prompt with strict json_schema response format and non-strict fallback. `eagle` compiles the schema into per-field FieldContracts and dispatches each as an independent LLM call under thread-pool concurrency of 4. `rsa` samples 12 initial candidates at temperature 0.8, then runs three merge rounds (~49 calls/instance).

VerbaNexAI Lab (Jesús Antonio Martínez Velandia, Jairo Enrique Serrano Castañeda, Edwin Alexander Puertas Del Castillo, Juan Carlos Martínez Santos, Universidad Tecnológica de Bolívar, Colombia) submitted three pipelines built around a domain-aware three-way router. `smart_orchestrator` inspects `metadata.domain` and dispatches to a RAG single-shot pipeline (medical/environmental/gen-

eral), to `parallel_extractor` (technical/legal), or to `precision_master` (zero-shot expert prompt engineering, all other domains).

8. Results

The official evaluation is complete. It was run by the organisers on the ainbox inference engine over both published models (Gemma 4 E4B and Qwen3-14B), complemented by team self-evaluations and by the June formal think-mode runs where a team lacked a clean spec-configuration (no-think) run — in each case we take a team's best *reported* 125-instance result regardless of thinking configuration (see §8.1). Scoring is on 125 instances: the full 145-instance gold test set less a 20-instance drop-set of schemas that could not be reliably evaluated on the inference stack (§8.2). The exclusion is applied uniformly to every system and to both baselines, so gap-closed comparisons remain fair.

8.1. Reference Baseline

The reference baseline is the official starter-kit `baseline` pipeline in no-think configuration, scored on the same 125 instances as every system. It was run three times, on different hardware, and the runs disagree slightly — Gemma {0.7739, 0.7780, 0.7895}, Qwen {0.7379, 0.7542, 0.7805}. We adopt the median run as the per-model floor — Gemma 4 E4B F1=0.7780 and Qwen3-14B F1=0.7542 — which is also the only run whose raw outputs were retained, so it is the only baseline we could independently re-score (it reproduced bit-for-bit). We deliberately avoid taking the maximum, which would set the least-reproducible, highest floor and understate every participant (see the reproducibility note in §9).

A methodological confound surfaced during evaluation and is worth recording as a finding in its own right. Both Gemma 4 E4B and Qwen3-14B support an extended-thinking mode, and teams ran in two incompatible configurations — thinking ON (slower, and on these schemas markedly *lower* structured-output F1) and no-think (faster, higher F1). On our evaluation-server run, the same `baseline` pipeline scored dramatically differently thinking off vs on (these two rows are one run in both modes, shown to isolate the mode effect; the no-think value here is the high end of the three-run spread above):

Model	Config	Baseline micro-F1
Gemma 4 E4B	no-think	0.7895
Gemma 4 E4B	thinking ON	0.4249
Qwen3-14B	no-think	0.7805
Qwen3-14B	thinking ON	0.7356

The Gemma 4 E4B swing (0.4249 thinking-ON vs 0.7895 no-think) is the most striking: on constrained structured extraction, extended thinking substantially *degrades* the model's schema conformance. This is consistent with reports that strict output-format constraints can interfere with reasoning (Tam et al. 2024), which in turn motivates reasoning before constraining (Banerjee et al. 2025) (§10). To keep gap-closed values comparable across teams we standardised the leaderboard on the no-think 125-instance baselines above; where a team's best reported result comes from a think-mode run, its F1 is still scored against the same no-think floor. An earlier organizer partial (30-instance slice, F1=0.8063) is retained here only as a historical artefact of the incremental sweep — it is not the final baseline.

8.2. Excluded Instances

Twenty of the 145 gold test instances were excluded uniformly from all systems and both baselines because their JSON schemas could not be reliably evaluated on the constrained-decoding inference stack. The exclusions fall into two structural failure classes:

- GBNF grammar-compilation failure (10 instances): the `medical_trials` (8) and `cultural_monuments` (2) schemas use `$defs`/`$ref` constructs that cause a llama.cpp GBNF grammar-

compilation failure (`400 failed to parse grammar`). These schemas are structurally unscoreable for any constrained-decoding system, including the reference baseline — the failure is at the grammar-compilation layer, before any generation occurs.
- Recursive-schema parser failure (10 instances): the `stem_biology` schema is a recursive `$ref` taxonomic-tree definition of arbitrary depth. It triggers parser recursion failures in participant systems (for example, DRILLER's `maximum recursion depth exceeded`).

Because the exclusion is applied identically to every system and to both no-think baselines, gap-closed comparisons on the remaining 125 instances remain fair. All figures in the rest of §8 are on these 125 instances.

8.3. Per-Pipeline Test-Set Results

The table below reports each team's best-performing pipeline per model on the 125 scored instances, with the thinking configuration of the reported cell. For each team we take the single best reported 125-instance result per model across any run — our evaluation or the team's own self-evaluation — regardless of thinking configuration (no-think = the task spec; think = the June formal run) and with no per-run reliability gate; several teams' best cell is a think-mode run (DRILLER's Gemma, SEsml's Qwen). A run's F1 already counts its failed or collapsed instances as misses, so each reported number is a faithful lower bound. F1 values are scored against the fixed no-think baselines (Gemma 0.7780, Qwen 0.7542).

Team	Gemma pipeline	Gemma F1	Gemma mode	Qwen pipeline	Qwen F1	Qwen mode
DRILLER	mixed-extractors-self-consistency-rag	0.8285	think	enriched-schema-rag	0.8346	nothink
Krishan	schema_dynamic	0.8142	nothink	schema_dynamic	0.8076	nothink
CodeStrange	combo_guard_react_repair_ref	0.8090	nothink	combo_guard_react_repair_ref	0.8039	nothink
FranRodrigo	cot	0.7812	nothink	cot	0.8081	nothink
SEsml	baseline	0.7746	nothink	adaptive	0.8017	think
VerbaNex	—	n/a	—	precision_master	0.8013	nothink
UC3M	prompted	0.7984	nothink	prompted	0.3629	nothink
GRADIANT	stable	0.7092	nothink	stable	0.7766	nothink
Inigo	e232	0.7504	nothink	e232	0.7320	nothink
JSONautas	grammar_cd	0.6139	nothink	prompt_eng	0.6534	nothink
MOLD	vigil	0.7354	nothink	arcane	0.7115	nothink

Complete per-pipeline results — all pipelines × both models on the 125 scored instances, with per-instance token and wall-time detail — are in the results artifact `results/2026-07-08-ainbox-missing-eval/`.

One cell is reported as `n/a` and warrants explanation. VerbaNex Gemma is unscoreable on our engine: all three VerbaNex Gemma pipelines reliably crash the ainbox Gemma backend at scale (each completes in isolation with a smoke F1 ≈ 0.76, but destabilises the shared engine over a full run). VerbaNex therefore receives gap 0 on Gemma and is carried entirely by its strong Qwen result; its true Gemma standing is likely higher than its leaderboard position reflects.

Two teams' best cells come from runs with per-instance reliability issues, which we surface rather than suppress. JSONautas Qwen (`prompt_eng`, F1=0.6534) had 5 instances that hard-failed on our inference backend (no output); these are counted as misses, so the true F1 is a lower bound. GRADIANT's best cells (`stable`, 0.7092 Gemma / 0.7766 Qwen) come from a higher-variance pipeline that collapses

to near-empty output on some instances (24 Gemma / 5 Qwen); its steadier `limited` pipeline scores 0.6792 / 0.7564. GRADIANT's own test-set reports reproduce our `stable` numbers exactly. In every case we report the highest-F1 cell, since the collapsed instances are already penalised within that F1.

8.4. Final Leaderboard

The official leaderboard ranks the eleven teams by average gap closed across the two published models, taking each team's best-performing pipeline per model and scoring against the fixed no-think baselines (Gemma 0.7780, Qwen 0.7542) on the 125 scored instances.

Rank	Team	Avg gap closed
1	DRILLER	0.2773
2	Krishan	0.1900
3	CodeStrange	0.1709
4	FranRodrigo	0.1171
5	SEsml	0.0965
6	VerbaNex	0.0958
7	UC3M	0.0461
8	GRADIANT	0.0455
9	Inigo	0.0000
9	JSONautas	0.0000
9	MOLD	0.0000

DRILLER tops the leaderboard, closing 27.7% of the baseline-to-perfect gap; the field spans from ~28% down to 0% (the three teams whose best pipeline did not exceed the baseline on either model receive gap 0, and rank equal-last). A paired-bootstrap test (§8.8) qualifies this order: the top three are statistically indistinguishable on a uniform re-run, so their 1–2–3 ranking is best read as a cluster above the baseline rather than a clean separation. VerbaNex's rank-6 standing is carried entirely by its Qwen result, since its Gemma pipelines are unscoreable on our engine (§8.3) — its true position is likely higher. UC3M is a mixed case: a strong Gemma cell (0.7984) paired with a collapsed Qwen cell (0.3629) that pulls its average down.

8.5. Per-Model Leaderboards

The averaged leaderboard hides substantial per-model variation. Ranking each published model on its own — each team's best pipeline, scored against that model's no-think baseline — reveals both a decisive overall winner and several model-specific reversals.

Gemma 4 E4B (baseline F1 0.7780):

Rank	Team	Best pipeline	F1	Gap closed
1	DRILLER	mixed-extractors-self-consistency-rag	0.8285[a]	0.2277
2	Krishan	schema_dynamic	0.8142[a]	0.1630
3	CodeStrange	combo_guard_react_repair_ref	0.8090[a]	0.1396
4	UC3M	prompted	0.7984	0.0921
5	FranRodrigo	cot	0.7812	0.0147

Five teams clear the Gemma baseline; five score below it — SEsml, Iñigo, MOLD, GRADIANT, and JSONautas — and VerbaNex has no scoreable Gemma cell (§8.3).

Qwen3-14B (baseline F1 0.7542):

Rank	Team	Best pipeline	F1	Gap closed
1	DRILLER	enriched-schema-rag	0.8346[a]	0.3270

Rank	Team	Best pipeline	F1	Gap closed
2	FranRodrigo	cot	0.8081	0.2194
3	Krishan	schema_dynamic	0.8076[a]	0.2170
4	CodeStrange	combo_guard_react_repair_ref	0.8039[a]	0.2022
5	SEsml	adaptive	0.8017	0.1930
6	VerbaNex	precision_master	0.8013	0.1916
7	GRADIANT	stable	0.7766	0.0909

Seven teams clear the Qwen baseline. DRILLER wins both models, and by a wider margin on Qwen (0.327) than on Gemma (0.228) — though a uniform paired-bootstrap re-run does not confirm that lead as significant (§8.8); the Qwen board is also deeper — seven teams above baseline versus five. The reversals are what the average hides: FranRodrigo ranks second on Qwen yet does not clear the Gemma baseline; UC3M is its mirror image, fourth on Gemma but collapsing on Qwen (F1 0.3629, a strict json-schema failure that voids most of its output); and VerbaNex scores only on Qwen because its pipelines crash our Gemma backend (§8.3). Reporting per model keeps a strong single-model result from being averaged into invisibility.

[a] On both models, DRILLER, Krishan, and CodeStrange form a statistical three-way tie under a uniform paired-bootstrap re-run (§8.8), despite their leaderboard order.

8.6. Strategy vs. Performance

The top of the leaderboard is a spread of distinct strategies rather than a single dominant recipe. DRILLER's winning system is a self-consistency RAG ensemble (`mixed-extractors-self-consistency-rag` on Gemma, `enriched-schema-rag` on Qwen) — the only true fan-out-and-vote pipeline in the field. Krishan's runner-up is a schema-adaptive single-call pipeline (`schema_dynamic`) that derives its labels at runtime from the schema enum order. CodeStrange's third-place system is a guard-first repair pipeline (`combo_guard_react_repair_ref`) that drafts, validates against a five-kind checker, and fires up to two repair calls on escalation.

What is notable is how tightly these top systems cluster on the clean 125 instances: all three land at roughly 0.80–0.83 F1 on both models (DRILLER 0.8285/0.8346, Krishan 0.8142/0.8076, CodeStrange 0.8090/0.8039). The gap-closed metric magnifies the small F1 differences among them because the baseline floor (0.75–0.78) is already high — a few points of F1 translate into large relative gap-closed swings. Below this cluster, FranRodrigo's chain-of-thought pipeline and SEsml's adaptive pipeline sit just above the baseline on one or both models. The three zero-gap teams (Inigo, JSONautas, MOLD) scored below the baseline on both models.

Joining each pipeline's strategy tags (§7.2) with its gap-closed exposes which choices actually pay off (Figure 3). Two single-pass levers dominate. Retrieval is the strongest: embedding-RAG pipelines average 0.073 gap closed, against 0.023 for no external knowledge and 0.012 for static example pools — roughly a 3× premium. Schema enrichment is next: enriched-schema pipelines average 0.043 versus 0.009 for raw schemas. Spending more *inference*, by contrast, does not pay: single-call pipelines average 0.044 gap closed, above both the 2–3-call (0.012) and 4+-call (0.030) groups, and by approach, chain-of-thought (0.053) and direct-prompt (0.037) beat multi-extractor (0.030) and agentic-loop (0.012). Self-consistency is mixed — the two vote pipelines (both DRILLER's) average 0.089, but verify-revise loops (0.017) trail no self-consistency at all (0.034). The recipe that emerges is a single well-prompted call with retrieval and an enriched schema — precisely the shape of the top three systems — while the multi-step agentic machinery several teams invested in did not, on this task, earn its cost (§9.3–§9.4). These readings are directional: n = 35 pipelines, uneven cell coverage, and the think/no-think mix all limit significance.

8.7. Efficiency: Gap Closed per Token

A token budget and a wall-time budget are part of the task (§5.4), so raw gap closed is only half the picture: a pipeline that buys a point of F1 with ten thousand extra tokens is not obviously better than a

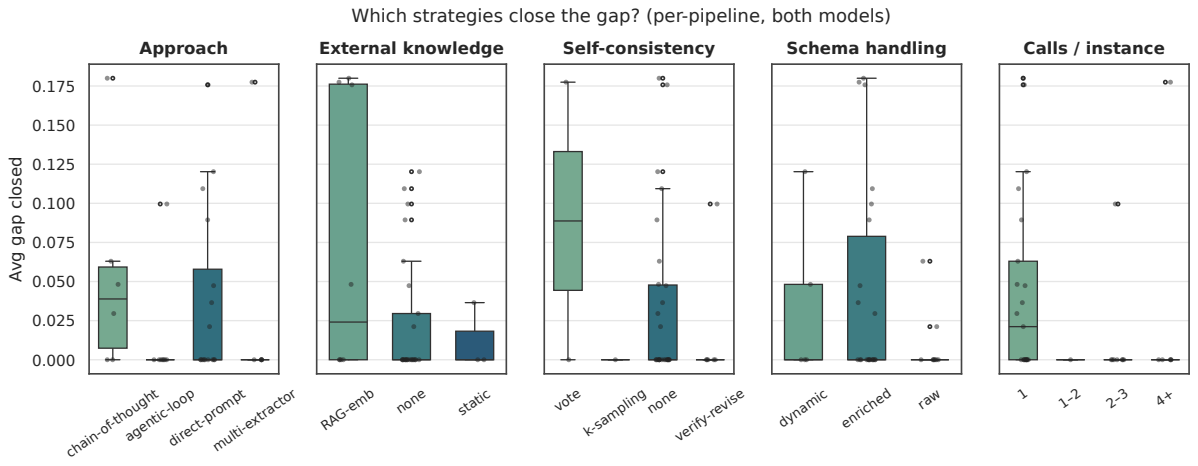

Figure 3: Gap-closed by strategy category across the five classification dimensions of §7.2 (one point per pipeline, best cell over both models). Retrieval and schema enrichment are the levers that pay off; additional inference passes are not.

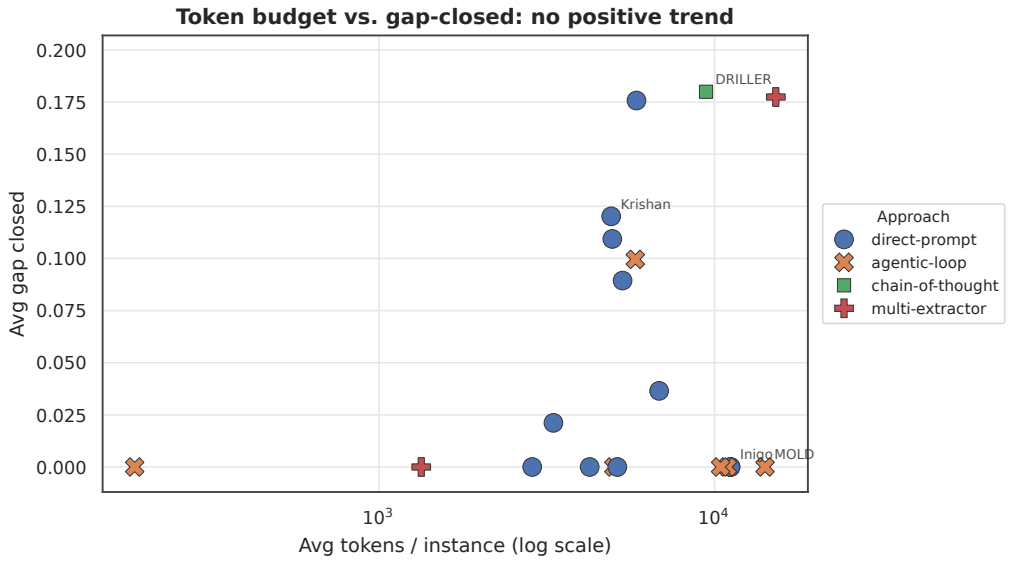

Figure 4: Average token budget versus average gap closed, one point per pipeline, marked by approach (log-scaled x-axis). There is no positive relationship — the most token-hungry pipelines sit at zero gap, and the best gap-per-token pipelines are mid-cost single calls.

cheaper one. For efficiency we search *every* instrumented pipeline of each team — not just its highest-gap one — and keep the configuration with the best gap-per-token, since a team's most cost-effective pipeline is frequently not the one it submitted for maximum score. All figures come from our uniform engine re-runs under identical conditions, so absolute gaps here run somewhat below §8.5 (which takes each team's best-reported cell across all sources, including self-evaluations) but are directly comparable for cost. Cells with no token instrumentation are omitted (FranRodrigo, VerbaNex, SEsml).

Rank	Team	Best gap-per-token pipeline	Gap closed	Tokens/inst.	Gap / 1K tok
1	DRILLER	enriched-schema-rag	0.180	~6.0K	0.0300
2	Krishan	schema_dynamic	0.123	~4.3K	0.0288
3	CodeStrange	combo_guard_react_repair_ref	0.107	~5.4K	0.0198
4	UC3M	prompted	0.042	~5.5K	0.0077

Searching over all pipelines sharpens the compute argument rather than softening it. DRILLER is the efficiency winner as well as the raw winner — but with a different pipeline than the one it submitted for the title. Its championship system is a four-trial self-consistency ensemble that averages ~15.9K tokens per instance for a gap of 0.185 (0.012 gap per 1K tokens); its own single-call `enriched-sche`

8.8. Statistical Significance

Because the top of the leaderboard is a spread of a few F1 points against a high baseline (§8.5), we test how much of the ordering is statistically real. We run a paired bootstrap over the 125 scored instances — 20,000 resamples, each recomputing pooled micro-F1 for every system on the *same* resampled instances — comparing each team's flagship pipeline to the baseline and to the other top systems. To keep the comparison fair, every system is scored on a single uniform re-run on our engine, without the best-of-configuration selection and think-mode boosts the official leaderboard permits (§8.3); absolute F1 here therefore runs slightly below §8.5, but all systems sit on identical footing.

Model	System	micro-F1	95% CI	vs. baseline
Gemma	Baseline	0.778	[0.756, 0.799]	—
Gemma	DRILLER	0.817	[0.791, 0.840]	$p = 0.002$
Gemma	Krishan	0.809	[0.786, 0.832]	$p = 0.001$
Gemma	CodeStrange	0.809	[0.786, 0.831]	$p < 0.001$
Qwen	Baseline	0.754	[0.729, 0.778]	—
Qwen	DRILLER	0.802	[0.775, 0.826]	$p = 0.002$
Qwen	Krishan	0.800	[0.777, 0.823]	$p < 0.001$
Qwen	CodeStrange	0.804	[0.780, 0.826]	$p < 0.001$

Two things follow. The top systems beat the baseline decisively: DRILLER, Krishan, and CodeStrange each exceed the reference baseline with $p \leq 0.002$ on both models, so the headline result — that small models meaningfully close the gap — is not a single-run artefact. But the top three are not separated from one another: every pairwise comparison within the trio is non-significant on both models (Gemma $p = 0.49$ / 0.43 / 0.97; Qwen $p = 0.90$ / 0.86 / 0.72, for DRILLER–Krishan / DRILLER–CodeStrange / Krishan–CodeStrange), and on the uniform Qwen run CodeStrange in fact edges DRILLER. The 1–2–3 order on the official leaderboard (§8.4) is produced by two amplifiers — gap-closed magnifies small F1 differences against the 0.75–0.78 baseline, and DRILLER's best-reported cells come from a think-mode Gemma run and a self-evaluation Qwen run (§8.3) — not by a per-model advantage that survives a like-for-like test. The honest reading is a statistical three-way tie at the top, comfortably above the baseline. The analysis is limited to the four systems with per-instance instrumented data on both models; the bootstrap script is released with the results (Appendix D).

8.9. A Note on Comparability

The numbers in this section are illustrative, not exactly reproducible, and should be read with that caution. Two effects move them beyond ordinary LLM stochasticity. First, the teams did not all run on the same hardware: our own uniform evaluation was blocked for several teams by container-implementation and dependency errors (§9.6), so some cells are teams' self-reported runs on their own machines, and hardware alone shifts model scores. Second, even the reference baseline varied across runs for the same reason — which is why we report the median of three runs (§8.1) rather than any single one. We therefore read the leaderboard as a comparison of strategies, not as an exact win/lose ordering; §9.5 argues that this is intrinsic to the shared-task setting rather than a weakness specific to GenSIE.

---

9. Discussion

GenSIE was designed not only to rank systems but to answer a set of concrete questions about how far small language models can be pushed on schema-guided extraction — and the diversity of the thirty-five submitted pipelines, spanning single-call prompting, retrieval augmentation, grammar-constrained decoding, verify-revise loops, and self-consistency ensembles, turns the leaderboard into a natural experiment for probing them. We organise the discussion around five questions. First, how much of the gap

between a small-model baseline and perfect extraction is actually closeable without fine-tuning, and where does the residual error concentrate (§9.1)? Second, does constraining the decoder to the schema grammar help, and at what cost (§9.2)? Third, does spending more inference compute — additional calls, ensembling, retrieval gates — buy accuracy (§9.3), and relatedly, do multi-step agentic loops beat a single well-prompted call, and on which instances (§9.4)? Fourth, are these results reproducible, given non-deterministic pipelines and a shared inference backend (§9.5)? And fifth, what did our evaluation harness cost us — was hosting inference centrally, over CPU-only participant containers, the right way to run a small-model shared task at all (§9.6)?

Because the field covers essentially the full strategy space, most of these questions can be read off cross-team and within-team contrasts rather than argued in the abstract. The insight we hoped to extract from a shared task — rather than from any single team's paper — is exactly this kind of controlled comparison: the same instance set, the same evaluator, and enough architectural variety that a strategy's effect is visible against its neighbours. The caveat throughout is that the comparison rests on a mix of thinking configurations and a 20-instance drop-set (§8), so where a cross-team contrast would be confounded we lean on within-team controlled comparisons (a team's ensemble versus its own single call, or a compute ladder within one submission).

9.1. The SLM-Frontier Gap Is Closeable but Not Eliminated

The headline result of GenSIE 2026 is that small language models in the 3B–14B parameter tier can close a meaningful fraction of the baseline-to-perfect error on zero-shot schema-guided extraction from Spanish text — but the gap is not eliminated. The best system (DRILLER) closes 27.7% of the baseline-to-perfect error on average across Gemma 4 E4B and Qwen3-14B, and the top three systems (DRILLER, Krishan, CodeStrange) form a tight cluster at roughly 0.80–0.83 F1 on both models over the clean 125 instances — a cluster that a paired-bootstrap test cannot statistically separate (§8.8). That is a substantive but bounded result: with an already-high baseline near 0.75–0.78, the best submissions pull perhaps a fifth of the way to perfect, and roughly four-fifths of the residual error is untouched. That residual is expected to concentrate in high-semantic-distance fields (score 4) and deeply nested schemas whose evidence is dispersed across sentences. It is also unevenly distributed across domains (Figure 5): the top systems clear the baseline comfortably on Stem, Cultural, and Technical schemas (median per-instance micro-F1 $\approx$ 0.85–0.90) but struggle on Research, Environmental, Legal, and General, where the per-instance distribution carries a heavy low tail and the median sits at or below the baseline.

This result is substantive because it is achieved without any fine-tuning — all systems operate under the no-training policy, relying entirely on prompt engineering, retrieval, schema enrichment, and structured-output constraints. The remaining gap motivates the next round: a harder, agentic formulation where the schema is not handed over directly.

9.2. Grammar-Constrained Decoding Is Double-Edged

The provisional signal from earlier self-evaluations — that grammar-constrained decoding (Willard and Louf 2023) (JSONautas `grammar_cd`) was a leading strategy — does not survive the official evaluation under a unified baseline. Against the fixed no-think floors on the 125 scored instances, JSONautas's best cells (`grammar_cd` on Gemma F1=0.6139, `prompt_eng` on Qwen F1=0.6534) both sit well below the baseline, and the team lands at rank equal-last with gap 0 — grammar-constrained decoding, while its best Gemma pipeline, does not clear the floor. The earlier double-digit gap-closed figure the team reported was an artefact of scoring against a depressed local baseline (their own Gemma baseline 0.4061; the thinking-ON reference baseline is 0.4249): a low floor manufactures a large numerator. Under the shared no-think baseline, grammar-constrained decoding no longer leads the field.

More pointedly, grammar-constrained decoding is *also the source* of the unscoreable-schema failures documented in §8.2: the llama.cpp GBNF compiler cannot compile the `$defs`/`$ref` schemas of the `medical_trials` and `cultural_monuments` domains, which forced their exclusion from scoring for every constrained-decoding system, including the baseline. Grammar constraints buy determinism

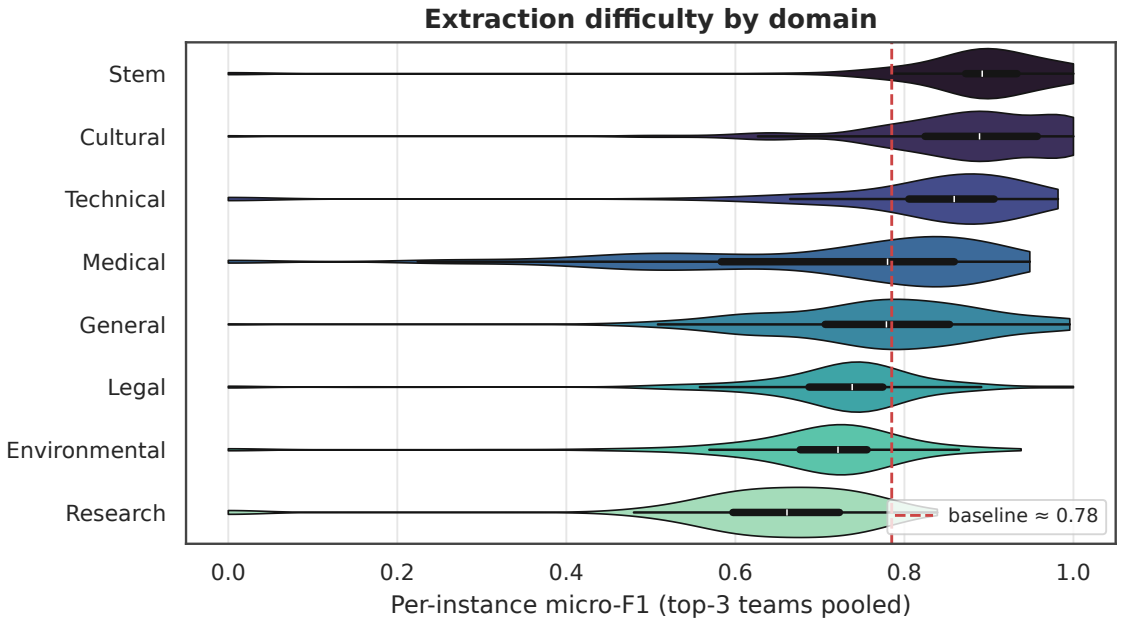

Figure 5: Per-instance micro-F1 by domain (top-three teams pooled, 125 scored instances), sorted by median; the dashed line is the mean no-think baseline. Extraction is markedly harder — and higher-variance — on Research, Environmental, Legal, and General schemas than on Stem or Cultural ones.

and guaranteed well-formedness on the schemas they *can* compile, but they fail hard — at compile time, before any generation — on the schema constructs they cannot. This is a genuinely double-edged finding: the same mechanism that enforces structure is the one that renders a fifth of the schemas (three of the sixteen) unscoreable.

9.3. Does More Compute Help?

More compute does not help — one of the clearest results of the official evaluation. DRILLER submitted the field's only true ensemble (`mixed-extractors-self-consistency-rag`: four interleaved internal extractions aggregated by a schema-aware self-consistency vote) alongside two single-call RAG pipelines. On the clean 125-instance test set in the spec (no-think) configuration, the ensemble is *worse* than DRILLER's own single-call `enriched-schema-rag` on both models: F1 0.8165 vs 0.8211 on Gemma 4 E4B, and 0.8129 vs 0.8346 on Qwen3-14B — a 2.2-point regression on Qwen for roughly triple the token budget. The self-consistency vote does not recover errors the single call makes; it mostly resamples the same mistakes. (DRILLER's leaderboard entry uses its Gemma ensemble cell only because that cell was measured in think mode, where it edges ahead; under a like-for-like no-think comparison the single call wins on both models.)

The MOLD team provides a within-team controlled experiment: four pipelines of systematically increasing compute (`baseline`→`mira`→`vigil`→`arcane`). On the test set they are statistically indistinguishable — Gemma/Qwen F1 of 0.735/0.710 (`vigil`), 0.734/0.712 (`arcane`), 0.730/0.704 (`mira`), 0.726/0.691 (`baseline`) — and *all four* sit below the reference baseline, so the added retrieval gates buy nothing. The one clear counter-example is CodeStrange, whose `combo_guard_react_repair_ref` (a guard-first pipeline with a ReAct repair loop) beats its lighter `combo_guard_list_spacy_ref` sibling by ~3.7 points on Gemma (0.809 vs 0.772). The pattern that emerges is that compute helps only when it is *targeted repair* of detected errors (CodeStrange), not blind resampling (DRILLER) or extra retrieval gates (MOLD).

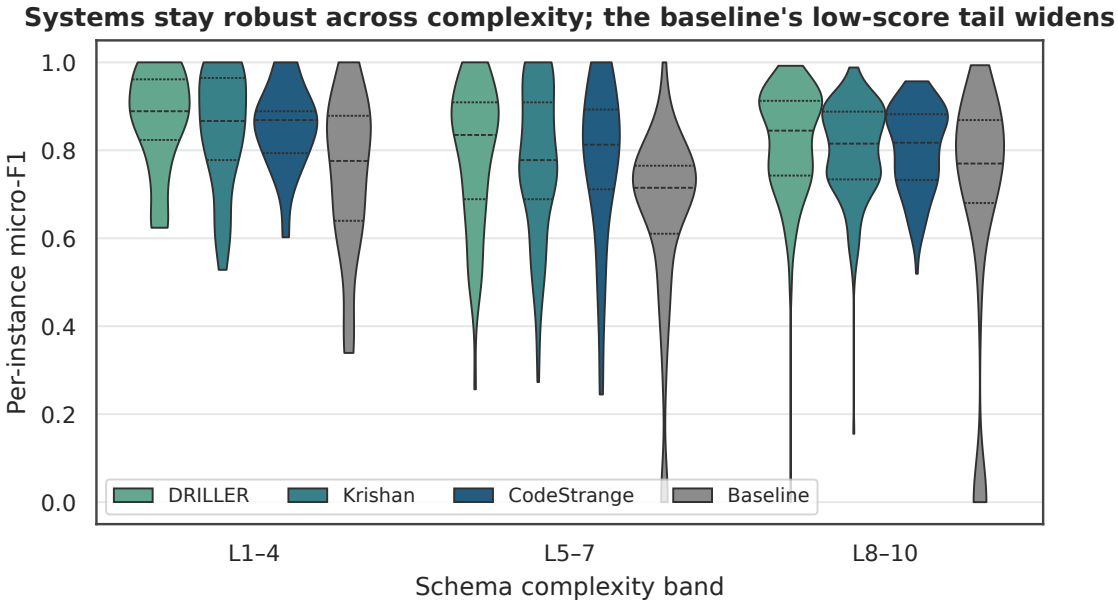

Figure 6: Per-instance micro-F1 distributions by complexity band for the top three teams and the baseline (best cell per team). Medians stay high and roughly flat across bands for the teams; the baseline develops a widening low-score tail, so the systems' advantage is concentrated in preventing catastrophic failures rather than in lifting the median.

9.4. Agentic Loops Versus One-Shot

Single-call pipelines with good retrieval or constrained decoding are competitive with — and often better than — multi-step agentic loops, consistent with §9.3. Breaking the results down by schema complexity sharpens the picture and exposes where the residual error lives. Against the per-bucket baseline, the top systems close a large fraction of the gap on low- and mid-complexity schemas but almost none on the hardest. DRILLER's best Gemma pipeline closes 45% of the gap on L1–L4 instances, 31% on L5–L7, and just 5% on L8–L10; Krishan's `schema_dynamic` shows the same collapse (20% / 34% / 4%), as does CodeStrange (41% / 24% / 2%). Over half of the scored set (68 of 125 instances) is L8–L10, and on that bulk every system is pinned within a few points of the baseline. Two caveats keep this honest. The gap-closed metric mechanically compresses on buckets where the baseline is already strong: the reference baseline scores 0.805/0.809 (Gemma/Qwen) on L8–L10 versus only 0.734/0.723 on L5–L7, so the small L8–L10 gap-closed reflects a near-ceiling baseline with little relative headroom left, not necessarily harder absolute extraction — in absolute F1 the systems' weakest band is actually mid-complexity L5–L7. What the breakdown does show cleanly is where the *agentic machinery* earns its keep: it converts headroom into gap-closed on the low-complexity instances and adds essentially nothing over the baseline on the high-complexity bulk. Whether those deep schemas are genuinely hard or the baseline already handles them, more inference passes do not move the needle.

Seen at the per-instance level (Figure 6), this is clearer still: the top systems hold a high median micro-F1 (≈ 0.85) across all three complexity bands — they do not collapse on hard schemas — while the baseline's distribution grows an increasingly heavy low-score tail as complexity rises. The systems' real advantage is in that tail: they prevent the catastrophic per-instance failures the baseline suffers on complex schemas, which the aggregate gap-closed metric compresses because the baseline's median stays high.

9.5. Variance and Reproducibility

Two reproducibility problems surfaced, one on the participant side and one on ours. On the participant side, non-determinism from undeclared sampling temperature is enough to make a pipeline unrankable:

one submission's single-call pipeline produced micro-F1 of 0.59, 0.60, and 0.35 across three runs on the same input — a 0.25-point swing — while the leaderboard is read off a single run per team. For next year we will require submissions to either set temperature=0 or report the mean and standard deviation across three runs, so that a ±0.125-variance pipeline is flagged differently from a ±0.001 one.

A third reproducibility concern is the ranking itself: we found (§8.8) that a paired-bootstrap test cannot statistically separate the top three systems, so the leaderboard's 1–2–3 order is an indicative cluster rather than a robust ordering — a caution we build into how the results are reported.

On our side, the shared inference engine was itself a reproducibility hazard. The self-hosted Gemma backend (llama.cpp) crashed under sustained load: some pipelines that ran cleanly in isolation destabilised the shared engine over a full 125-instance run, and a single participant's traffic could bring the backend down at scale — silently mixing one team's failure into another's results before we caught and repaired it (a per-instance re-run on a restarted engine). The one Gemma cell we could not stabilise (VerbaNex) is reported as `n/a` rather than silently zero-filled (§8.3). This is not merely an operational nuisance; as we argue next, it is a symptom of a harness design we now think was the wrong call.

The deepest consequence is that the results are not fully comparable across teams, and we want to state this plainly rather than bury it. Because our uniform run was blocked for several teams, some cells are teams' own runs on their own hardware, and hardware differences shift model scores beyond ordinary sampling noise — the same `baseline` pipeline scored between 0.774 and 0.790 (Gemma) and between 0.738 and 0.781 (Qwen) across three machines (§8.1). We made an extraordinary effort to normalise this — a single hosted engine, a fixed model set, a uniform drop-set, and a median baseline — and still could not make it airtight. We do not regard this as a weakness specific to GenSIE. It is intrinsic to the shared-task format itself: historically, IberLEF and NLP shared tasks hand each participant an unannotated test set, and each runs its system — and often its own baseline — on whatever hardware it has before submitting. That format inherently yields results that are neither perfectly reproducible nor perfectly comparable, because outcomes depend not only on model quality, data quality, and prompt quality but on *where the computation runs*. Every paper that compares algorithms, metaheuristics, or models across machines carries the same caveat, usually implicitly; we prefer to make it explicit. The practical implication is how the leaderboard should be read: the point of GenSIE is not who won and who lost, but which strategies worked and why. Several teams that finished below the baseline nonetheless contributed genuinely interesting designs — grammar-constrained decoding, domain-routed retrieval, guard-and-repair loops — that are well worth analysing, reimplementing, and re-running under stronger and more uniform compute to see what they can actually yield. That analytic payoff, not the exact ordering, is what we hope the community takes from this round.

9.6. The Cost of a Hosted-Inference Harness

The most instructive lesson of GenSIE 2026 is not about any one submission but about the evaluation harness itself. We required every team to ship a CPU-only container that our organiser-hosted inference server drove over an OpenAI-compatible endpoint, and we hosted the models ourselves — a fixed set of published and held-out small models — so that no team could tune to a single target and cross-model robustness would be rewarded. That decision was well-intentioned, and, in hindsight, the source of most of the friction in the task.

Two kinds of difficulty followed. Operationally, a containerised-agent-plus-hosted-inference harness generates a whole class of avoidable failures — dependencies resolved at container startup rather than build time, `/info` healthchecks that never come up, context-window budgets exceeded, pipelines that never register — and keeping a single shared multi-model engine stable and *fair* under many teams' heterogeneous traffic proved genuinely hard (§9.5). Container-side friction is largely preventable: a mandatory CI smoke run (one `/info` and one `/run` before acceptance) would have caught most of it days before the deadline, and we will require it. The engine-side instability, however, is not something a submission check can fix — it is inherent to sharing one inference backend across many uncontrolled workloads.

More fundamentally, hosting inference ourselves flattens the design space we most wanted teams

to explore. A team that must call our server cannot fine-tune, cannot distil a task-specific student, cannot bring its own small or quantised model, and cannot choose or optimise its inference stack — precisely the techniques that make small-model deployment an interesting engineering problem in the first place. The comparability that a fixed hosted model set buys comes at the price of forbidding the very engineering that an edge-deployment setting is about; and normalising across models is in any case hard and imperfect (the thinking-mode baseline shift of §8.1 is one symptom of that difficulty).

For the next round we are inverting the architecture. The organisers will host an MCP server that exposes the task — its instructions, schemas, and evaluation resources — as tools and resources, and teams will run inference on their own infrastructure with whatever techniques they choose: fine-tuning, distillation, custom small models, their own serving stack and optimisations. This trades away perfect cross-model normalisation for a far larger and more realistic design space, and it moves the hardest operational burden off the organisers and onto infrastructure each team already controls. We think that trade is worth making; it is the direction §10 sketches.

10. Related Work

Schema- and instruction-guided extraction with LLMs. Guiding large language models to produce structured output is an active line. GoLLIE (Sainz et al. 2024) shows that conditioning on annotation guidelines improves zero-shot extraction; InstructUIE (Xiao Wang et al. 2023) and KnowCoder (Li et al. 2024) unify heterogeneous IE tasks through instruction tuning and code-schema representations respectively; ADELIE (Qi et al. 2024) aligns LLMs specifically for extraction; and a recent survey (Xu et al. 2024) maps the generative-IE landscape. GenSIE differs from this line along several axes at once. It targets Spanish rather than the English-centric mainstream; it constrains systems to the small-model (3B–14B), CPU-deployable tier under a no-training policy, so the available levers are prompting, retrieval, and constrained decoding rather than fine-tuning; it scores against held-out models of several families to penalise single-model tuning; and it couples the JSON-schema target with sentence-level grounding annotations and an explicit null-as-ground-truth contract that charges parametric hallucination. To our knowledge it is the first IberLEF shared task in this schema-guided-extraction-with-SLMs setting.

Format constraints and reasoning. Our thinking-mode result (§8.1) — that Gemma 4 E4B's extended-thinking configuration sharply lowers structured-output F1 — sits alongside a growing literature on the interaction between output-format constraints and reasoning. Tam et al. (Tam et al. 2024) report that requiring strict formats can degrade LLM performance relative to free-form generation, and work on constrained decoding (Banerjee et al. 2025) argues that grammar constraints and chain-of-thought reasoning can interfere, motivating designs that reason before constraining. We observe the same tension from the benchmark side: think-mode both depresses the baseline and, through grammar-constrained decoding, is implicated in the unscoreable-schema failures of §8.2. Our contribution is not the mechanism but its magnitude on a concrete Spanish extraction benchmark and its consequence for fair cross-team comparison.

Normalized metrics and abstention. The gap-closed metric (§5.2) is a normalized-improvement score: it reports the fraction of the baseline-to-ceiling headroom a system closes, the same algebraic form as human-normalized scores in reinforcement learning (Mnih et al. 2015) and the human-baseline-gap framing of general-purpose benchmarks such as SuperGLUE (Wang et al. 2019). Our specific construction — normalising per model and averaging over several held-out model families in an extraction shared task — is, as far as we know, novel in this setting, but the shape is not, and we present it as an adaptation rather than a new metric. Finally, the null-as-ground-truth contract connects GenSIE to work on abstention and unanswerable questions (Kirichenko et al. 2025): returning `null` for an unsupported field is a field-level abstention, and rewarding it directly is what deters the parametric-hallucination failure mode the benchmark is built to expose.

11. Conclusion

GenSIE @ IberLEF 2026 set out to test how far small language models in the 3B–14B parameter range can be pushed on schema-guided information extraction from Spanish text, under a CPU-only deployment contract and a multi-model evaluation that penalises single-model tuning. Eleven teams submitted thirty-five pipelines across the full range of strategies the task admits. The 271-instance training/dev silver set spans eight of the benchmark's nine domains (Research is test-only), with grounding coverage of 94%; the 145-instance gold test set comprises 145 instances balanced across seen-schema and unseen-schema partitions.

The official evaluation, scored on 125 gold instances after a uniform 20-instance drop-set, was topped by DRILLER, whose RAG pipeline family closes 27.7% of the baseline-to-perfect error averaged across Gemma 4 E4B and Qwen3-14B — its self-consistency ensemble leading on Gemma (F1=0.8285, think mode) and its single-call variant leading on Qwen (F1=0.8346). A tight cluster of top systems — DRILLER, Krishan's schema-adaptive `schema_dynamic` (19.0%), and CodeStrange's guard-first `comb o_guard_react_repair_ref` (17.1%) — lands at roughly 0.80–0.83 F1 on the clean instances, demonstrating that the SLM tier is meaningfully capable of schema-guided extraction in Spanish without any fine-tuning, though most of the residual error remains. Two methodological findings frame that headline. First, Gemma 4 E4B's thinking mode dramatically shifts the baseline (F1=0.4249 thinking-ON vs ~0.78 no-think on the 125 scored instances), so a unified no-think floor was needed to compare teams fairly; scored against that floor, grammar-constrained decoding no longer leads. Second, grammar-constrained decoding is double-edged — it is also the reason a fifth of the schema space (the `$defs`/`$ref` and recursive schemas) could not be scored at all. The residual error concentrates, as expected, in high-semantic-distance fields and deeply nested schemas at L8–L10.

The next round we are scoping moves the task in a different direction: a fully agentic challenge in which participants build autonomous agents that solve opaque tasks by exploring an organiser-hosted MCP server with tools and resources, rather than extracting structured output from a fixed text. The schemas, grounding annotations, and evaluator developed for GenSIE are released under the Creative Commons Attribution 4.0 licence and archived on Zenodo (see *Data and Code Availability*); we invite the community to build on them, and to bring their critiques of the dataset, the evaluator, and the gap-closed metric to the IberLEF 2026 session (Bonet-Jover et al. 2026).

12. Data and Code Availability

Data. The GenSIE benchmark — the schema catalogue, grounding annotations, the silver development instances, and the gold test set — is released under the Creative Commons Attribution 4.0 International (CC BY 4.0) licence and archived on Zenodo under DOI 10.5281/zenodo.21299326 (concept DOI, resolving to the latest version; this v1.0 release is 10.5281/zenodo.21299327). Every source document carries its provenance — URL, retrieval date, and licence — in an `index.json` record; the corpus draws only on freely licensed Spanish sources (Wikipedia, Wikinoticias, the Boletín Oficial del Estado, the CIMA medicines registry, open-access scientific outlets, and a small synthetic-raw complement, §4.1).

Code. The task harness, the reference baseline, the evaluator, and the scripts that produce every table and figure in this paper are in the `gia-uh/gensie-internal` repository, at the tag `iberlef-202 6-camera-ready` (commit pinned in the camera-ready). The frozen final leaderboard (`LEADERBOARD.m d`) and its recomputation script reproduce the reported gap-closed values exactly; the full reproduction procedure is in Appendix D.

13. Funding and Conflicts of Interest

Funding. *(To be completed in the camera-ready: funding bodies and grant numbers.)*

Conflicts of interest. GenSIE is organised by members of GIA (University of Havana) and GPLSI/CENID (University of Alicante). Four of the eleven ranked teams are affiliated with the Univer-

sity of Havana, including the top-ranked team (DRILLER). To limit the effect of this, the leaderboard is produced by a single automated procedure applied uniformly to every team (Appendix D), the drop-set and per-model baselines are applied identically to all systems and both baselines (§8.1–§8.2), and the metric together with its per-pipeline inputs is released for independent recomputation. We further stress that the top-three cluster is separated by only one to two F1 points and that the ranking fuses organiser re-runs, team self-evaluations, and think/no-think configurations; a paired-bootstrap analysis (§9.5) finds ranks two and three statistically indistinguishable and the leader's margin significant only on Qwen. We therefore report the leaderboard as an indicative ordering rather than a statistically separated ranking, and we caution against over-reading small differences near the top.

14. Acknowledgements

We thank the eleven participating teams for their submissions and their patience through a demanding evaluation window, and the IberLEF 2026 organising committee for hosting the task.

15. Declaration on Generative AI

The GenSIE task and this overview made substantial, disclosed use of generative AI, and we report it here in the interest of full transparency. (1) *System and dataset:* the benchmark is built by a semi-automatic agentic curation pipeline (§4) in which large-language-model agents perform source acquisition, silver-instance generation, grounding, and ensemble auditing, with three human gate points; this agent involvement is a described contribution of the work rather than an incidental aid. (2) *Evaluation:* the reference baseline and the evaluation harness call large language models by construction (§5–§6). (3) *Preparation of this manuscript:* during the preparation of this work the author(s) used Claude and GPT-class assistants for drafting, paraphrasing and reformulation, results tabulation and numerical consistency-checking, and for writing the evaluation and analysis scripts. All reported results, claims, and their interpretation were reviewed and verified by the authors, who take full responsibility for the content of this publication.

16. References

Banerjee, Debangshu, Tarun Suresh, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. "CRANE: Reasoning with Constrained LLM Generation." *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Bonet-Jover, Alba, José Ángel González-Barba, and Luis Chiruzzo. 2026. "Overview of IberLEF 2026: Natural Language Processing Challenges for Spanish and Other Iberian Languages." *Proceedings of the Iberian Languages Evaluation Forum (IberLEF 2026), Co-Located with the 42nd Conference of the Spanish Society for Natural Language Processing (SEPLN 2026), CEUR-WS.org*.

Gemma Team, Google DeepMind. 2026. "Gemma 4 Technical Report." *arXiv Preprint arXiv:2607.02770*.

Honnibal, Matthew, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. *spaCy: Industrial-Strength Natural Language Processing in Python*. https://doi.org/10.5281/zenodo.1212303.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2021. "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data* 7 (3): 535–47.

Kirichenko, Polina, Mark Ibrahim, Randall Balestriero, et al. 2025. "AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions." *arXiv Preprint arXiv:2506.09038*.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*.

Li, Zixuan, Yutao Zeng, Yuxin Zuo, et al. 2024. "KnowCoder: Coding Structured Knowledge into LLMs for Universal Information Extraction." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 8758–79.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. 2014. "GloVe: Global Vectors for Word Representation." *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Qi, Yunjia, Hao Peng, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. 2024. "ADELIE: Aligning Large Language Models on Information Extraction." *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Qwen Team. 2025. "Qwen3 Technical Report." *arXiv Preprint arXiv:2505.09388*.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Sainz, Oscar, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. "GoLLIE: Annotation Guidelines Improve Zero-Shot Information-Extraction." *The Twelfth International Conference on Learning Representations (ICLR)*.

Tam, Zhi Rui, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. "Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models." *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 1218–36.

Wang, Alex, Yada Pruksachatkun, Nikita Nangia, et al. 2019. "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems." *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Wang, Wenhui, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." *Advances in Neural Information Processing Systems (NeurIPS)*.

Wang, Xiao, Weikang Zhou, Can Zu, et al. 2023. "InstructUIE: Multi-Task Instruction Tuning for Unified Information Extraction." *arXiv Preprint arXiv:2304.08085*.

Wang, Xuezhi, Jason Wei, Dale Schuurmans, et al. 2023. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." *International Conference on Learning Representations (ICLR)*.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*.

Willard, Brandon T., and Rémi Louf. 2023. "Efficient Guided Generation for Large Language Models." *arXiv Preprint arXiv:2307.09702*.

Xiao, Shitao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. "C-Pack: Packed Resources for General Chinese Embeddings." *arXiv Preprint arXiv:2309.07597*.

Xu, Derong, Wei Chen, Wenjun Peng, et al. 2024. "Large Language Models for Generative Information Extraction: A Survey." *Frontiers of Computer Science* 18 (6): 186357.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*.

---

17. Appendices

17.1. Schema Catalogue

The benchmark defines the schema classes below across nine domains. The paper's headline count of sixteen schemas refers to the primary extraction targets of the development set; the full catalogue here (43 classes) additionally includes the test-set schemas and the nested component schemas that larger targets embed (for example `TrialArm` within `ClinicalTrialRecord`, or `Taxon` within the recursive `TaxonomicClassification`). Complexity is tagged per schema on five dimensions — structural Depth, Semantic Distance, information Dispersion, constraint Rigidity, and Grounding difficulty (each 1–4) — and summarised as an Overall level (L1–L10). The machine-readable source of truth is the `@complexity(...)` decorator on each class under `schemas/{domain}/`.

Schema	Domain	Subdomain	Depth	SemDist	Disp	Rig	Grd	Overall
`AlbumReview`	Cultural	Album Review	4	4	4	2	3	L10
`LiteraryWork`	Cultural	Literature	1	2	1	2	1	L2
`MediaReview`	Cultural	Media	2	3	2	3	2	L5
`MusicAlbum`	Cultural	Media	2	1	2	2	1	L2
`MonumentBIC`	Cultural	Monuments	1	2	1	3	2	L3
`ArtistProfile`	Cultural	People	1	2	2	2	3	L4
`TheaterPlaybill`	Cultural	Theater	3	2	3	3	3	L7
`EnvironmentalRuling`	Environmental	Assessments	2	4	3	2	2	L7
`NamedEntities`	General	Entities	3	3	1	3	1	L5
`TextAnswer`	General	Extraction	1	1	1	1	1	L1
`NewsArticle`	General	News	2	2	2	2	2	L4
`CourtCaseTimeline`	Legal	Case Timeline	3	4	4	3	3	L8
`ContractSummary`	Legal	Contracts	2	3	3	4	3	L5
`LegalInquiry`	Legal	Legislation	2	4	3	3	2	L8
`LegislativeAct`	Legal	Legislation	2	3	4	3	2	L7
`CourtRulingHeader`	Legal	Rulings	1	2	1	3	1	L3
`Penalty`	Legal	Rulings	1	1	1	2	1	L2
`SentencingLogic`	Legal	Rulings	3	3	3	3	2	L7

Schema	Domain	Subdomain	Depth	SemDist	Disp	Rig	Grd	Overall
RecipeProcedural	Lifestyle	Recipes	4	4	3	4	3	L10
Activity	Lifestyle	Travel	1	2	2	3	2	L2
ItineraryDay	Lifestyle	Travel	2	1	3	2	1	L3
TravelItinerary	Lifestyle	Travel	3	2	4	2	2	L5
DischargeSummary	Medical	Discharge	3	3	4	3	3	L8
DiseaseProfile	Medical	Diseases	3	2	2	2	2	L5
DosageInstructions	Medical	Drugs	3	4	3	4	3	L9
DrugDescription	Medical	Drugs	3	3	3	3	2	L8
SideEffect	Medical	Drugs	1	3	2	3	2	L3
ClinicalTrialRecord	Medical	Trials	2	2	3	4	3	L7
TrialArm	Medical	Trials	1	1	2	2	1	L2
PaperArgument	Research	Paper Argument	4	4	4	3	3	L10
CelestialObjectProperties	Stem	Astronomy	2	3	3	4	3	L5
Taxon	Stem	Biology	4	1	1	3	1	L3
TaxonomicClassification	Stem	Biology	4	2	2	3	2	L6
ChemicalIdentity	Stem	Chemicals	1	2	1	3	1	L3
SafetySheet	Stem	Chemicals	2	3	2	3	2	L6
ObservationLog	Stem	Observations	3	3	4	3	3	L8
ChemicalSafety	Stem	Safety	2	2	2	3	2	L4
APIEndpoint	Technical	Api	3	2	3	4	2	L7
ChangelogHighlights	Technical	Releases	2	2	2	2	2	L4
SoftwareReleaseHeader	Technical	Releases	1	2	1	3	1	L3
SecurityIncident	Technical	Security Incident	4	4	4	4	3	L9
SoftwareDescription	Technical	Software	2	3	3	3	3	L9
SecurityVulnerability	Technical	Vulnerabilities	1	2	2	3	2	L4

17.2. Sample Gold Instance

Domain: Medical · Subdomain: Trials · Schema: `ClinicalTrialRecord` (L7)

Input Context: > “El ensayo clínico aleatorizado de Fase 3 de la vacuna mRNA-1273 (Moderna) evaluó a 30,420 participantes. Los resultados primarios mostraron una eficacia del 94.1%…”

Input Instruction: > “Extrae el nombre de la medicación y el resultado del ensayo, siendo positivo si

es mayor de 90% de efectividad, negativo si es menor del 70%, inconcluso en cualquier otro caso."

Target Schema (excerpt):

```
{
  "type": "object",
  "properties": {
    "medication_name": { "type": "string" },
    "clinical_outcome": {
      "type": "string",
      "enum": ["POSITIVE", "NEGATIVE", "INCONCLUSIVE"],
     "description": "Infer the semantic success of the trial per the instruction rule"
    }
  }
}
```

Gold Output:

```
{ "medication_name": "mRNA-1273 (Moderna)", "clinical_outcome": "POSITIVE" }
```

Grounding: - `medication_name` — *extractive*; quote "vacuna mRNA-1273 (Moderna)". - `clinical_outcome` — *inferred*; reasoning "94.1% > 90% → POSITIVE per the instruction's threshold rule". This field exercises the semantic-distance dimension: the value is not a span but a rule-based classification over an extracted figure.

17.3. Per-Team `/info` Payloads

Every participant container exposes a `GET /info` endpoint that the evaluator calls before running any instance, returning the team's declared pipelines. Three representative payloads follow (verbatim for JSONautas; reconstructed from the container's declared pipelines for the others). The complete set of eleven payloads is archived alongside the run artifacts.

```
{
  "team_name": "JSONautas",
 "institution": "University of Holguín - University of Oriente - University of Computer Science",
  "pipelines": [
    { "name": "prompt_eng",  "description": "Standard prompt-based extraction." },
    { "name": "rag",         "description": "RAG-based extraction." },
   { "name": "grammar_cd",  "description": "Extraction based on Grammar-Constrained Decoding." }
  ]
}

{
  "team_name": "DRILLER",
  "institution": "Universidad de La Habana",
  "pipelines": [
   { "name": "enriched-schema-rag",             "description": "Single-call enriched extraction wit
   { "name": "enriched-inline-reasoning-rag",      "description": "Chain-of-thought wrapper over th
   { "name": "mixed-extractors-self-consistency-rag", "description": "Four-trial heterogeneous ense
  ]
}

{
  "team_name": "CodeStrange",
```

```
  "institution": "Universidad de La Habana / independent",
  "pipelines": [
   { "name": "combo_guard_react_repair_ref", "description": "Guard-first draft, five-kind validato
   { "name": "combo_guard_list_spacy_ref",   "description": "Per-field role routing through a Spanis
   { "name": "grounded",                "description": "fastembed retrieval over truncated source ta
  ]
}
```

17.4. Reproduction Notes

The canonical artifacts are in `gia-uh/gensie-internal` under `results/`. The gold test set is `data/test_gold/` (145 instances); the 20-instance drop-set (§8.2) is excluded at scoring time, leaving 125 scored instances.

Official test-set leaderboard (final).

1. Serve the two published models on an OpenAI-compatible endpoint (Gemma 4 E4B and Qwen3-14B).
2. For each team container, run `gensie eval --data data/test_gold/ --pipeline <P> --model <M> --url <container>` (a no-think proxy forces the spec configuration); this writes a per-`(team, pipeline, model)` report with per-instance TPS, `system_keys`, `gold_keys`, timing, and token usage. The organiser runtime that orchestrates this (per-team proxy, boot quirks, idempotent skip) is in `results/2026-07-08-ainbox-missing-eval/_runtime/`.
3. Recompute the gap-closed leaderboard with the drop-set applied: `results/2026-07-08-ainbox-missing-eval/leaderboard-final.py` reads the reports, excludes the drop-set instances, recomputes per-model micro-F1 (`precision = ∑tps/∑system_keys`, `recall = ∑tps/∑gold_keys`), and averages `max(0, (F1−F1_base)/(1−F1_base))` over the two models against the no-think baselines (Gemma 0.7780, Qwen 0.7542).

The organiser re-run reports are in `results/2026-07-08-ainbox-missing-eval/reports/`, and the frozen leaderboard is `LEADERBOARD.md` in the same folder; the final gap-closed values in `LEADERBOARD.md` recompute exactly from these inputs. The paired-bootstrap significance analysis (§9.5) is reproducible via `significance-bootstrap.py` in the same folder. Some per-pipeline cells were sourced from team self-evaluations and June think-mode snapshots (§7.3, §8.3); the complete per-pipeline provenance is included in the archived data release (see *Data and Code Availability*).

Development dry run (historical). The preliminary 40-instance starter-kit sanity check on `llama-3.2-3b-instruct` is reproducible via the harness in `results/2026-05-26-starter-dry-run/`; those numbers are illustrative and do not feed the official leaderboard.

This paper. The CEURART PDF is regenerated from `drafts/2026-06-29-paper-v2.md` with the no-install toolchain documented in `drafts/render-ceurart/` (quarto-bundled pandoc + tectonic + `ceurart.cls`, citations via `references.bib`).